

Acceso a Datos

RA1 · Práctica 2

Cambio de almacenamiento y migración a JSON

Departamento de Informática

Curso 2026–2027

Índice

Punto de partida y objetivo	2
1. Conservar la versión inicial	2
2. Sustituir la persistencia CSV por JSON	2
3. Definir el formato JSON	3
4. Implementar la clase traductora	3
Reglas de protección de datos	4
5. Ejemplos de comportamiento	4
Migración correcta	4
Destino existente	5
6. Registrar y analizar los cambios en Git	5
7. Entrega y evaluación	5
Rúbrica de evaluación	5
8. Autorreflexión	6
Inventario de clases afectadas	6
Reflexión sobre el diseño	7

Punto de partida y objetivo

Esta práctica representa el **15 % de la calificación del RA1** de la asignatura.

Partiendo del código desarrollado en la primera práctica de gestión de la gasolinera, crea una versión que almacene los clientes y los pagos en **JSON en lugar de CSV**.

A continuación, se presenta un ejemplo de fichero en formato JSON.

```
{
  "clientes": [
    {
      "id": 1,
      "nombre": "Ana López",
      "telefono": "600123456",
      "matricula": "1234ABC"
    }
  ]
}
```

Los saltos de línea son innecesarios. Están incluidos por facilitar la lectura humana, pero el formato JSON ignora los saltos de línea. Es decir, podría ir todo en la misma línea, un único cliente por línea...

Las funcionalidades, las validaciones y el comportamiento de la aplicación deben mantenerse. El objetivo es comprobar cómo responde tu diseño a un cambio de formato y analizar qué partes del programa necesitan modificarse.

Además, implementarás una **clase traductora** que lea los CSV de la primera versión y migre sus datos al nuevo formato. Tanto el desarrollo como el análisis de los cambios deberán quedar documentados en el repositorio Git.

1. Conservar la versión inicial

Antes de modificar el código, comprueba que la primera versión funciona y que sus fuentes están guardadas en un commit. Anota el identificador de ese commit: será la referencia para comparar ambas versiones.

Trabaja según la organización de tu repositorio:

- **Un repositorio por proyecto:** crea una rama nueva, por ejemplo, `practica2-json`, a partir del commit de la primera práctica. Desarrolla la segunda versión en esa rama.
- **Un repositorio para toda la asignatura:** crea una carpeta nueva, por ejemplo, `practica2-json/`, con una copia de los fuentes necesarios de la primera práctica. Conserva la carpeta original. Registra la copia inicial en un commit independiente, antes de introducir cambios, para poder comparar esa carpeta consigo misma en los commits posteriores.

Documenta la rama o las carpetas utilizadas y los commits de referencia.

2. Sustituir la persistencia CSV por JSON

La nueva versión deberá mantener las mismas funcionalidades de la P1, a las que añadimos las siguientes:

1. Guardar en JSON cada alta y pago **antes de confirmar su éxito**.

2. Informar de forma controlada de errores de acceso o JSON malformado, sin sobrescribir silenciosamente los datos afectados.
3. Ofrecer, en el arranque, la posibilidad de migrar los datos de unos archivos en CSV que se encuentren en el directorio indicado por consola.

La ejecución habitual de esta versión utilizará únicamente JSON como almacenamiento. La lectura de CSV quedará reservada a la migración (aunque debe mantenerse el código de escritura de CSV).

3. Definir el formato JSON

Puedes utilizar un fichero conjunto o ficheros separados. Documenta:

- Los nombres y la ubicación de los ficheros.
- La estructura JSON y el significado y tipo de cada campo.
- La representación de fechas y cantidades decimales.
- La representación del almacenamiento vacío. Por ejemplo, ¿Cómo es el fichero cuando no hay clientes?

Se recomienda una estructura como la siguiente (se muestra solo el archivo de clientes):

```
{
  [
    {"id": 1,"nombre": "Ana López","telefono": "600123456","matricula": "1234ABC"},
    {"id": 2,"nombre": "Óscar López","telefono": "654321098","matricula": "1234CBC"},
    {"id": 3,"nombre": "Antonio López","telefono": "600123456","matricula": "1234DDD"}
  ]
}
```

Pero otra estructura posible (en un único archivo) podría ser:

```
{
  "clientes": [
    {
      "id": 1,
      "nombre": "Ana López",
      "telefono": "600123456",
      "matricula": "1234ABC"
    }
  ],
  "pagos": [
    {
      "id": 1,
      "clienteId": 1,
      "fecha": "2026-09-11",
      "importe": 40.50,
      "litros": 25.00,
      "combustible": "Gasolina 95"
    }
  ]
}
```

4. Implementar la clase traductora

Implementa una clase `MigraCSVToJson.java` con la responsabilidad explícita de traducir los datos CSV de la primera versión al formato JSON elegido.

La migración será una operación explícita, separada de las operaciones habituales de gestión. En el primer arranque de la aplicación, antes de mostrar el menú normal y cuando todavía no exista el almacenamiento JSON, se preguntará si hay ficheros CSV que migrar. Si la respuesta es afirmativa, se solicitarán las rutas de los ficheros CSV de clientes y de pagos y se ejecutará la migración sobre esos ficheros. Solo después de una migración completada podrá continuar la ejecución normal. Si la respuesta es negativa, la aplicación continuará directamente con la ejecución normal y creará el almacenamiento JSON vacío que corresponda.

En los arranques posteriores, cuando ya exista el almacenamiento JSON, no se volverá a preguntar por la migración: la aplicación comenzará directamente con el menú normal.

Si se solicita una migración y se produce un error, se informará del motivo, no se generará un resultado parcial y no se continuará como si la migración se hubiera completado.

La migración deberá informar del número de clientes y pagos migrados, y de la ubicación del resultado.

Reglas de protección de datos

- Los CSV originales no se modificarán ni eliminarán.
- Si falta un fichero de origen requerido, la operación fallará con un mensaje claro.
- Si se encuentra algún dato inválido, se cancelará toda la migración y se indicará el fichero, el registro o línea afectada y el motivo. No se omitirán registros silenciosamente ni se publicará una migración parcial.
- El destino debe estar libre: si ya existe cualquiera de los ficheros de salida previstos, la migración se rechazará sin sobrescribirlos ni añadir registros. Esto también se aplica a JSON vacíos creados previamente por la aplicación.
- Si falla la escritura, se informará del error y no se anunciará la migración como completada.

Para repetir una prueba, utiliza un destino nuevo.

No se exige fusionar datos, resolver conflictos automáticamente ni recuperar una operación interrumpida por un corte de corriente. Hazlo sencillo y simple.

5. Ejemplos de comportamiento

Migración correcta

Con Ana López (cliente 1) y su pago 1 de los ejemplos de la primera práctica como únicos registros en los CSV de origen, y un destino libre:

```
Origen CSV: datos-practica1/
Destino JSON: datos-practica2/
Validación completada.
Clientes migrados: 1
Pagos migrados: 1
Migración completada en datos-practica2/.
```

Al iniciar la aplicación JSON sobre ese destino:

```
Opción: 2
ID  NOMBRE      TELÉFONO  MATRÍCULA
1   Ana López   600123456 1234ABC

Opción: 5
ID  CLIENTE     FECHA      IMPORTE  LITROS  COMBUSTIBLE
1   Ana López   11/09/2026 40.50 €  25.00    Gasolina 95
```

Destino existente

Migración cancelada: el destino ya contiene datos.json.
Selecciona un destino libre. **No** se han modificado los ficheros existentes.

6. Registrar y analizar los cambios en Git

Realiza commits cuando vayas terminando funcionalidades. Los mensajes deben ser comprensibles para permitir seguir el cambio de formato, la implementación de la traducción y las comprobaciones.

Los archivos de datos, excepcionalmente, deben estar versionados en Git.

Incluye en la entrega un fichero **CAMBIOS.md** con las decisiones de diseño no impuestas en la práctica que hayas decidido.

7. Entrega y evaluación

Entrega en el repositorio:

1. La primera versión conservada y la segunda versión en la rama o carpeta indicada.
2. El código Java de la aplicación JSON y de la clase traductora.
3. Un **README.md** actualizado con instrucciones de compilación, ejecución, dependencias, formato JSON y procedimiento de migración.
4. El informe **CAMBIOS.md**, con inventario, recuento, evidencias de Git y reflexión.
5. Las pruebas y los datos ficticios necesarios para reproducir los resultados.

Rúbrica de evaluación

La práctica se califica sobre 10 puntos. Se evaluarán las evidencias del código, su ejecución y el historial de Git.

Criterio	Puntuación	Evidencias y valoración
1. Funcionamiento con JSON y conservación del comportamiento anterior	3 puntos	La versión JSON conserva las funcionalidades y reglas de validación de P1: altas, listados, búsquedas, registro y consulta de pagos, relaciones entre pagos y clientes y continuación de identificadores. Se crea el almacenamiento en el primer arranque, se recupera al reiniciar. El JSON es válido, documentado y conserva tipos, valores, identificadores y relaciones sin ambigüedad ni pérdida de precisión. Ante un JSON malformado o un error de acceso, se informa de forma controlada sin sobrescribir los datos afectados.
2. Traducción correcta y protección de los datos durante la migración	3 puntos	En el primer arranque, cuando no existe el destino JSON, la aplicación ofrece la migración y solicita las rutas solo si se acepta. La clase traductora lee el CSV real de P1 y genera un JSON que la aplicación puede utilizar directamente. Conserva todos los registros, campos, identificadores y relaciones; admite el almacenamiento vacío documentado; e informa de los recuentos. No modifica los CSV ni sobrescribe un destino ocupado. Ante un origen ausente, un dato inválido o un fallo de escritura, informa de la causa sin publicar una migración parcial.
3. Cada alta o pago se guarda antes de confirmarse	2 puntos	La aplicación escribe guarda el alta de cada cliente y de cada pago en el momento en el que se produce, antes de confirmar su éxito. Las demás funcionalidades leen los archivos para poder acceder a los datos actualizados.
4. Modularidad, poco acoplamiento, legibilidad y comentarios útiles	1 puntos	Existe una interfaz o abstracción común de lectura y escritura, con implementaciones concretas para CSV y JSON. El conocimiento de cada formato está localizado en esas implementaciones: la interfaz de usuario, las validaciones y la gestión no acumulan condicionales de formato ni duplican lógica de persistencia. Se valoran responsabilidades claras, legibilidad, comentarios útiles, gestión adecuada de recursos y una solución proporcional. Los cambios en las clases existentes son admisibles cuando estén justificados en CAMBIOS.md.
5. Historial Git e informe de cambios verificable	1 puntos	El historial permite comparar el punto de partida y la versión final mediante commits comprensibles. Los mensajes de los commits son claros. Hay, al menos, 3 commits.

Si se entrega dentro de las **dos horas indicadas**, la calificación obtenida se multiplicará por **1,2**. La calificación máxima tras aplicar este multiplicador será de **12 puntos**.

8. Autorreflexión

Esta es una sección añadida para cuando tengas tiempo.

Inventario de clases afectadas

Completa una tabla como esta con los nombres reales de tu solución:

Clase o interfaz	Situación	Cambio realizado	Motivo
Nombre real	Modificada / nueva / eliminada / renombrada	Descripción concreta	Por qué fue necesario

Indica por separado:

- Número de clases o interfaces existentes que has modificado.
- Número de clases o interfaces nuevas.
- Número de clases o interfaces eliminadas.
- Renombrados o movimientos, identificando origen y destino y si también cambió el contenido.
- Archivos de configuración, dependencias y documentación afectados, sin contarlos como clases.

Reflexión sobre el diseño

Responde con referencias a tu propio código:

1. ¿Dónde se conocía el formato CSV en la primera versión?
2. ¿Qué cambios fueron necesarios para utilizar JSON y cuáles introdujiste únicamente para permitir la migración?
3. ¿Tuviste que modificar el menú, las validaciones o las operaciones con clientes y pagos? Explica por qué, tanto si la respuesta es sí como si es no.
4. ¿Qué partes permanecieron intactas y qué decisiones del diseño lo hicieron posible?
5. ¿Qué cambiarías ahora para facilitar una futura sustitución del formato?

No se premia ocultar cambios ni concentrar toda la aplicación en una clase para reducir el recuento. El número de clases afectadas es una evidencia para analizar el acoplamiento, no una calificación automática.